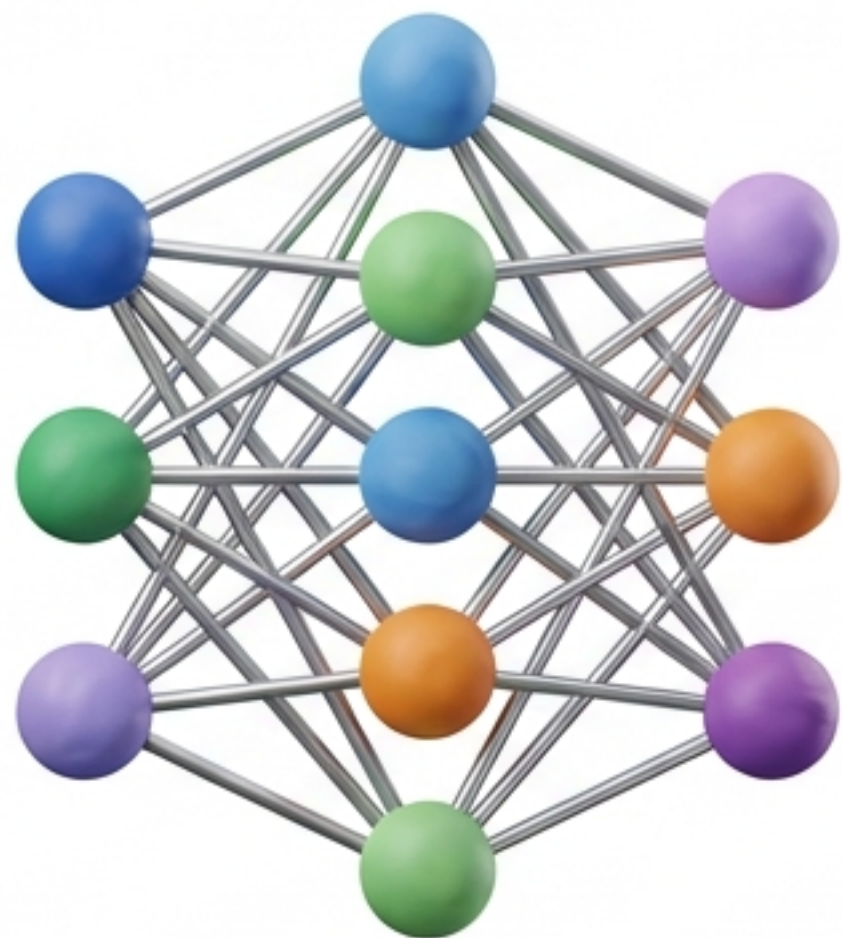




Tự động phân biệt với TensorFlow GradientTape

Thực tế, Framework hiện đại như TensorFlow tự động hóa toàn bộ việc Lan truyền ngược bằng Automatic Differentiation.

```
1 import tensorflow as tf
2
3 x = tf.zeros(shape =()) # Gia tri khoi tạo
4 with tf.GradientTape () as tape:
5     # Tape (Bang ghi) se lưu lại do thi cac phép toan
6     y = 2 * x + 3
7
8 # Yeu cau Tape tinh gradient cua y doi voi x (dao ham)
9 grad_of_y_wrt_x = tape.gradient(y, x)
```

Thực hành (Từ đầu): NaiveDense

Hãy xây dựng thủ công một lớp Dense sử dụng tensor ops.

```
1 import keras
2 from keras import ops
3
4 class NaiveDense:
5     def __init__(self, input_size, output_size,
6         activation=None):
7         self.activation = activation
8         self.W = keras.Variable(shape=(input_size, size),
9             initializer="uniform")
10        self.b = keras.Variable(shape=(output_size, ),
11            initializer="zeros")
12
13    def __call__(self, inputs): # Thuc hien Forward Pass
14        x = ops.matmul(inputs, self.W) + self.b
15        if self.activation:
16            x = self.activation(x)
17        return x
```




Thực hành (Từ đầu): Vòng lặp Huấn luyện

Khái quát lại quá trình **model.fit()** bằng đoạn code sau:

```
1 def one_training_step(model , images_batch , labels_batch):
2     with tf.GradientTape () as tape:
3         # 1. Forward pass de du doan va tinh Loss
4         predictions = model(images_batch)
5         loss = ops.sparse_categorical_crossentropy(labels_batch , predictions)
6         average_loss = ops.mean(loss)
7
8         # 2. Backward pass (Tinh gradient tu dong nho Tape)
9         gradients = tape.gradient(average_loss , model.weights)
10
11        # 3. Cap nhat trong so bang Optimizer (nhu SGD)
12        update_weights(gradients , model.weights)
13
14    return average_loss
```


Tổng kết Chương 2



Tensors: Nền tảng cấu trúc dữ liệu của học sâu. Bao gồm các bậc 0D, 1D, 2D, 3D, 4D v.v.



Phép toán Tensor: Phép nhân ma trận, phép cộng, ReLU... là những biến đổi hình học tinh vi giúp gỡ rối đa diện dữ liệu.



Trọng số (Weights): Tham số học hỏi chứa toàn bộ "kiến thức" của mô hình mạng.



Loss function & Optimizers: Loss đo lường độ chính xác (mục tiêu cần tối thiểu hóa). Optimizer như SGD giảm Loss thông qua đi ngược Gradient.



Backpropagation: Sử dụng Đồ thị tính toán và Quy tắc chuỗi để tính đạo hàm ngược một cách nhanh chóng qua cơ chế phân biệt tự động (GradientTape).